Embabel Agent Framework: Zero-to-Hero Developer Documentation

Version: 0.3.5-SNAPSHOT
Last Updated: March 2026
Target Audience: Java Developers building AI Agents

Table of Contents

---

1. Introduction to Embabel

1.1 What is Embabel?

Embabel (Em-BAY-bel) is a sophisticated framework for authoring agentic AI flows on the JVM that seamlessly mix LLM-prompted interactions with code and domain models. Built with **Kotlin** and **Spring Framework**, it supports intelligent path finding towards goals using advanced planning algorithms.

From the creator of Spring Framework, Embabel brings enterprise-grade patterns to AI agent development, making it possible to build production-ready agentic applications that integrate seamlessly with existing JVM systems.

1.2 Why Embabel?

Unlike traditional agent frameworks that rely on finite state machines or sequential execution, Embabel introduces **true planning capabilities** using Goal-Oriented Action Planning (GOAP). This enables agents to discover novel solutions by combining existing actions in ways you never programmed.

Key Differentiators

Feature	Description
Sophisticated Planning	Goes beyond simple workflows with dynamic, non-LLM AI planning using GOAP
Superior Extensibility	Add capabilities without editing existing code through dynamic planning
Strong Typing	Full object orientation with refactoring support and domain model behavior
Platform Abstraction	Clean separation between programming model and platform internals
LLM Mixing	Easy integration of different models for cost-effective, capable solutions
Spring Foundation	Built on familiar enterprise patterns with full DI and testing support
Designed for Testability	Both unit testing and agent end-to-end testing are easy from the ground up

1.3 Agentic AI Concepts

Agent An **Agent** in the Embabel framework is a self-contained component that bundles together domain logic, AI capabilities, and tool usage to achieve a specific goal on behalf of the user. Inside, it exposes multiple `@Action` methods, each representing discrete steps the agent can take.

Tools **Tools** extend the raw capabilities of an LLM by letting it interact with the outside world. On its own, a language model can only generate responses from its training data. Tools bridge the gap between text prediction and real-world action.

MCP (Model Context Protocol) **MCP** is a standardized way of hosting and sharing tools. Unlike plain tools, which are usually wired directly into one agent or app, an MCP Server makes tools discoverable and reusable across models and runtimes.

DICE (Domain Integrated Context Engineering) **DICE** enhances context engineering by grounding both LLM inputs and outputs in typed domain objects. Instead of untyped prompts, context is structured with business-aware models that provide precision, testability, and seamless integration.

---

2. Getting Started

2.1 Prerequisites

- **Java 21+** (Oracle or OpenJDK)
- **Maven 3.9+** or Gradle
- **API Key** from OpenAI, Anthropic, Google, or local LLM setup (Ollama/LM Studio)

2.2 Quick Start Options

Option A: Using Project Creator (Recommended)

```
# Install uv if not already installed
```

```
pip install uv

# Create a new project
uvx --from git+https://github.com/embabel/project-creator.git project-creator
```

Choose **Java** or **Kotlin** when prompted, specify your project details, and you'll have a working agent in under a minute!

Option B: Clone Examples Repository

```
git clone https://github.com/embabel/embabel-agent-examples
cd embabel-agent-examples/scripts/java
./shell.sh
```

Option C: Use GitHub Templates

- **Java Template**: github.com/embabel/java-agent-template
- **Kotlin Template**: github.com/embabel/kotlin-agent-template

2.3 Maven Configuration

Add the Embabel Agent Spring Boot starter to your `pom.xml`:

```
<properties>
    <embabel-agent.version>0.3.5-SNAPSHOT</embabel-agent.version>
</properties>

<dependencies>
    <!-- Shell Starter - Interactive command-line interface -->
    <dependency>
        <groupId>com.embabel.agent</groupId>
        <artifactId>embabel-agent-starter-shell</artifactId>
        <version>${embabel-agent.version}</version>
    </dependency>

    <!-- OpenAI Support -->
    <dependency>
        <groupId>com.embabel.agent</groupId>
        <artifactId>embabel-agent-starter-openai</artifactId>
        <version>${embabel-agent.version}</version>
    </dependency>
</dependencies>
```

2.4 Environment Setup

Create a `.env` file or set environment variables:

```
export OPENAI_API_KEY="your_openai_api_key_here"
export ANTHROPIC_API_KEY="your_anthropic_api_key_here"
export GOOGLE_API_KEY="your_google_api_key_here"
```

2.5 Your First Agent

```
package com.example.agent;

import com.embabel.agent.api.annotation.Agent;
import com.embabel.agent.api.annotation.Action;
import com.embabel.agent.api.annotation.AchievesGoal;
import com.embabel.agent.api.common.OperationContext;

@Agent(description = "A simple greeting agent")
public class GreetingAgent {

    public record UserInput(String content) {}
    public record Greeting(String message) {}

    @Action
    public Greeting greet(UserInput userInput, OperationContext context) {
        String response = context.ai()
            .withDefaultLlm()
            .generateText("Generate a friendly greeting for: " + userInput.content());
        return new Greeting(response);
    }
}
```

Run the shell and test:

```
./scripts/shell.sh
x "Say hello to the world"
```

---

3. Core Concepts

3.1 The Agent Architecture

Embabel models agentic flows in terms of:

- **Actions**: Steps an agent takes - the building blocks of agent behavior
- **Goals**: What an agent is trying to achieve
- **Conditions**: Evaluations while planning - reassessed after each action
- **Domain Model**: Objects underpinning the flow and informing Actions, Goals, and Conditions

3.2 Type-Driven Flow

Embabel uses **type signatures** to automatically infer execution plans:

```
@Agent(description = "Find news based on a person's star sign")
public class StarNewsFinder {

    @Action
    public StarPerson extractStarPerson(UserInput userInput, OperationContext context) {
        return context.ai()
            .withLlm(OpenAiModels.GPT_41)
```

```
            .createObject("Extract name and star sign from: " + userInput.getContent(),
                        StarPerson.class);
    }

    @Action
    public Horoscope retrieveHoroscope(StarPerson starPerson) {
        // Uses regular Spring service - not LLM
        return new Horoscope(horoscopeService.dailyHoroscope(starPerson.sign()));
    }

    @AchievesGoal(description = "Write an amusing writeup")
    @Action
    public Writeup writeup(StarPerson person, RelevantNewsStories stories,
                           Horoscope horoscope, OperationContext context) {
        // Final action that achieves the goal
        return context.ai()
            .withLlm(LlmOptions.withCriteria(ModelSelectionCriteria.getAuto())
                               .withTemperature(0.9))
            .createObject("Write something amusing...", Writeup.class);
    }
}
```

Inferred Execution Plan:

```
UserInput → extractStarPerson() → StarPerson → retrieveHoroscope() → Horoscope
→ findNewsStories() → RelevantNewsStories → writeup() → Writeup (Goal Achieved)
```

3.3 The Blackboard Pattern

The **Blackboard** serves as the shared memory system that maintains state throughout agent process execution:

```
// Objects are automatically added when returned from actions
@Action
public Person extractPerson(UserInput input, OperationContext context) {
    Person person = context.ai().createObject("Extract person from: " + input, Person.class);
    // person is automatically added to blackboard
    return person;
}

// Objects are retrieved by type for subsequent actions
@Action
public Greeting greetPerson(Person person, OperationContext context) {
    // Person is automatically injected from blackboard
    return new Greeting("Hello, " + person.name());
}
```

Key Characteristics:

- **Central Repository**: Stores all domain objects, intermediate results, and process state
- **Type-Based Access**: Objects are indexed and retrieved by their types

- **Ordered Storage**: Objects maintain the order they were added, with latest being default
- **Immutable Objects**: Once added, objects cannot be modified (new versions can be added)

3.4 Agent Process Lifecycle

An **AgentProcess** maintains state throughout its execution:

State	Description
`NOT_STARTED`	The process has not started yet
`RUNNING`	The process is executing without any known problems
`COMPLETED`	The process has completed successfully
`FAILED`	The process has failed and cannot continue
`TERMINATED`	The process was killed by an early termination policy
`KILLED`	The process was killed by the user or platform
`STUCK`	The process cannot formulate a plan to progress
`WAITING`	The process is waiting for user input or external event
`PAUSED`	The process has paused due to scheduling policy

---

4. Building Your First Agent

4.1 The @Agent Annotation

The `@Agent` annotation defines a class as an agent. It is a Spring stereotype annotation, so it triggers Spring component scanning:

```
@Agent(description = "Agent that writes and reviews stories")
public class WriteAndReviewAgent {
    // Actions go here
}
```

Required Parameters:

- `description`: Human-readable description used by the LLM in agent selection

Optional Parameters:

- `planner`: Planning strategy (GOAP, UTILITY, SUPERVISOR)

4.2 The @Action Annotation

The `@Action` annotation marks methods that perform actions within an agent:

```
@Action(
    description = "Extract person information from user input",
    pre = {"condition1", "condition2"},  // Additional preconditions
    post = {"condition1", "condition2"}, // Additional postconditions
    canRerun = false,    // Whether action can be rerun
```

```
    readOnly = false,    // Whether action has no side effects
    clearBlackboard = false, // Whether to clear blackboard after execution
    cost = 0.0,          // Relative cost (0-1)
    value = 0.0          // Relative value (0-1)
)
public Person extractPerson(UserInput input, OperationContext context) {
    // Action implementation
}
```

4.3 The @AchievesGoal Annotation

The `@AchievesGoal` annotation marks an action as completing the agent's objective:

```
@AchievesGoal(description = "Review a story and provide feedback")
@Action
public ReviewedStory reviewStory(Story story, OperationContext context) {
    // Final action that achieves the goal
}
```

4.4 Complete Example: WriteAndReviewAgent

```
@Agent(description = "Agent that writes and reviews stories")
public class WriteAndReviewAgent {

    public record Story(String text) {}
    public record ReviewedStory(Story story, String review) {}

    @Action
    public Story writeStory(UserInput userInput, OperationContext context) {
        return context.ai()
            .withLlm(LlmOptions.withAutoLlm().withTemperature(0.8))
            .createObject("""
                You are a creative writer who aims to delight and surprise.
                Write a story about: %s
                """.formatted(userInput.getContent()), Story.class);
    }

    @AchievesGoal(description = "Review a story")
    @Action
    public ReviewedStory reviewStory(Story story, OperationContext context) {
        String review = context.ai()
            .withLlmByRole("reviewer")  // Uses configured reviewer LLM
            .generateText("""
                You are a meticulous editor.
                Carefully review this story: %s
                """.formatted(story.text()));
        return new ReviewedStory(story, review);
    }
}
```

4.5 Running Your Agent

```
# Set your API key
export OPENAI_API_KEY="your_key_here"

# Run the shell
./scripts/shell.sh

# Execute the agent
x "Tell me a story about a robot learning to paint"
```

---

5. Actions and Goals

5.1 Action Parameters

Action methods can have parameters from two categories:

Domain Objects: Automatically resolved from the blackboard

```
@Action
public Output process(Customer customer, Order order, OperationContext context) {
    // Customer and Order are resolved from blackboard
}
```

Infrastructure Parameters: Provided by the framework

```
@Action
public Output process(UserInput input, OperationContext context, Ai ai) {
    // OperationContext provides access to blackboard and LLM
    // Ai provides direct LLM access
}
```

5.2 Conditional Actions with SpEL

Use Spring Expression Language (SpEL) for dynamic preconditions:

```
@Action(pre = {"spel:assessment.urgency > 0.5"})
public void handleUrgentIssue(Issue issue, IssueAssessment assessment) {
    // Only runs when urgency exceeds 0.5
}

@Action(pre = {"spel:issueAssessment.urgency > 0.0"})
public void escalateUrgentIssue(GHIssue issue, IssueAssessment issueAssessment) {
    // Only runs for urgent issues
}
```

5.3 Reactive Triggers

The `trigger` field enables reactive behavior:

```
@Agent(description = "Chat message handler")
public class ChatAgent {
```

```
    @AchievesGoal(description = "Respond to user message")
    @Action(trigger = UserMessage.class)
    public Response handleMessage(UserMessage message, Conversation conversation) {
        return new Response("Received: " + message.content());
    }
}
```

5.4 Dynamic Cost Computation

Compute action costs dynamically at planning time:

```
@Agent(description = "Processor with dynamic cost")
public class DataProcessor {

    @Cost(name = "processingCost")
    public double computeProcessingCost(@Nullable LargeDataSet data) {
        if (data != null && data.size() > 1000) {
            return 0.9;  // High cost for large datasets
        }
        return 0.1;  // Low cost for small datasets
    }

    @Action(costMethod = "processingCost")
    public ProcessedData process(RawData input) {
        return new ProcessedData(input.transform());
    }
}
```

5.5 The @Provided Annotation

Access Spring beans from state classes using `@Provided`:

```
@EmbabelComponent
public class ReservationFlow {
    private final BookingService bookingService;

    public ReservationFlow(BookingService bookingService) {
        this.bookingService = bookingService;
    }

    @State
    public record CollectDetails(String customerId) {
        @Action
        public ConfirmReservation confirm(ReservationDetails details,
                                          @Provided ReservationFlow flow) {
            var booking = flow.bookingService.reserve(details);
            return new ConfirmReservation(booking);
        }
    }
}
```

6. Tools and MCP Integration

6.1 Creating Tools with @LlmTool

Annotate methods with `@LlmTool` to expose them to LLMs:

```
public class MathTools {

    @LlmTool(description = "Add two numbers together")
    public double add(@LlmTool.Param(description = "First number") double a,
                      @LlmTool.Param(description = "Second number") double b) {
        return a + b;
    }

    @LlmTool(description = "Multiply two numbers")
    public double multiply(double a, double b) {
        return a * b;
    }
}
```

6.2 Using Tools in Actions

```
@Action
public CalculationResult calculate(UserInput input, OperationContext context) {
    return context.ai()
        .withDefaultLlm()
        .withToolObject(new MathTools())  // Add tool object
        .createObject("Calculate: " + input.getContent(), CalculationResult.class);
}
```

6.3 Tool Groups

Tool groups organize related tools and can be conditionally exposed:

```
@Configuration
public class ToolGroupsConfiguration {

    @Bean
    public ToolGroup webToolsGroup(List<McpSyncClient> mcpSyncClients) {
        return new McpToolGroup(
            CoreToolGroups.WEB_DESCRIPTION,
            "docker-web",
            "Docker",
            Set.of(ToolGroupPermission.INTERNET_ACCESS),
            mcpSyncClients,
            callback -> {
                String name = callback.getToolDefinition().name();
                return (name.contains("brave") || name.contains("fetch"))
                       && !name.contains("batch");
            }
```

```
        );
    }
}
```

6.4 MCP Server Integration

Configure MCP servers in `application.yml`:

```
spring:
  ai:
    mcp:
      client:
        type: SYNC
        stdio:
          connections:
            brave-search-mcp:
              command: docker
              args: [run, -i, --rm, -e, BRAVE_API_KEY, mcp/brave-search]
              env:
                BRAVE_API_KEY: ${BRAVE_API_KEY}
```

6.5 Framework-Agnostic Tool Interface

Create tools programmatically:

```
// Simple tool with no parameters
Tool greetTool = Tool.create("greet", "Greets the user",
    (input) -> Tool.Result.text("Hello!"));

// Tool with parameters
Tool addTool = Tool.create("add", "Adds two numbers together",
    Tool.InputSchema.of(
        Tool.Parameter.integer("a", "First number"),
        Tool.Parameter.integer("b", "Second number")
    ),
    (input) -> Tool.Result.text("42")
);

// Typed tool with automatic schema generation
Tool addTool = Tool.fromFunction("add", "Adds two numbers",
    AddRequest.class, AddResult.class,
    input -> new AddResult(input.a() + input.b())
);
```

6.6 Subagent: Agent Handoffs as Tools

Delegate work to other agents using `Subagent`:

```
@Action
public Concert assembleConcert(ConcertPlan plan, OperationContext context) {
    return context.ai()
        .withDefaultLlm()
```

```
        .withTool(Subagent.ofClass(PerformanceFinder.class)
                          .consuming(WorksToFind.class))
        .creating(Concert.class)
        .fromPrompt("Assemble a concert based on: " + plan);
}
```

6.7 Progressive Tools with UnfoldingTool

Progressive tools enable dynamic tool disclosure:

```
// Create inner tools
Tool queryTool = Tool.create("query_table", "Execute a SQL query", ...);
Tool insertTool = Tool.create("insert_record", "Insert a new record", ...);
Tool deleteTool = Tool.create("delete_record", "Delete a record", ...);

// Create UnfoldingTool facade
var databaseTool = UnfoldingTool.of("database_operations",
    "Use this tool to work with the database. Invoke to see specific operations.",
    List.of(queryTool, insertTool, deleteTool));
```

---

7. Working with LLMs

7.1 LlmOptions Configuration

The `LlmOptions` class specifies which LLM to use and its hyperparameters:

```
// Create LlmOptions with model and temperature
var options = LlmOptions.withModel(OpenAiModels.GPT_4O_MINI)
                        .withTemperature(0.8);

// Use different hyperparameters for different tasks
var analyticalOptions = LlmOptions.withModel(OpenAiModels.GPT_4O_MINI)
                                  .withTemperature(0.2)
                                  .withTopP(0.9);
```

Important Methods:

- `withModel(String)`: Specify the model name
- `withRole(String)`: Specify the model role (configured in application.yml)
- `withTemperature(Double)`: Set creativity/randomness (0.0-1.0)
- `withTopP(Double)`: Set nucleus sampling parameter
- `withPersona(String)`: Add a system message persona

7.2 PromptRunner API

All LLM calls should be made via the `PromptRunner` interface:

```
@Action
public Story createStory(UserInput input, OperationContext context) {
```

```
    // Get PromptRunner with default LLM
    var runner = context.ai().withDefaultLlm();

    // Get PromptRunner with specific LLM options
    var customRunner = context.ai()
        .withLlm(LlmOptions.withModel(OpenAiModels.GPT_4O_MINI)
                           .withTemperature(0.8));

    return customRunner.createObject("Write a story about: " + input.getContent(),
                                      Story.class);
}
```

7.3 Core PromptRunner Methods

Object Creation:

- `createObject(String, Class<T>)`: Create a typed object from a prompt
- `createObjectIfPossible(String, Class<T>)`: Try to create an object, return null on failure
- `generateText(String)`: Generate simple text response

Tool and Context Management:

- `withToolGroup(String)`: Add tool groups for LLM access
- `withToolObject(Object)`: Add domain objects with @LlmTool methods
- `withPromptContributor(PromptContributor)`: Add context contributors
- `withImage(AgentImage)`: Add an image for vision-capable LLMs

7.4 Working with Images

```
var image = AgentImage.fromFile(imageFile);
var answer = context.ai()
    .withLlm(AnthropicModels.CLAUDE_35_HAIKU)
    .withImage(image)
    .generateText("What is in this image?");
```

7.5 Model Configuration in application.yml

```
embabel:
  models:
    default-llm: gpt-4o-mini
    default-embedding-model: text-embedding-3-small
    llms:
      cheapest: gpt-4o-mini
      best: gpt-4o
      reasoning: o1-preview
    embedding-services:
      fast: text-embedding-3-small
      accurate: text-embedding-3-large
```

7.6 Supported LLM Providers

Provider	Starter Dependency	Configuration
OpenAI	`embabel-agent-starter-openai`	`OPENAI_API_KEY`
Anthropic	`embabel-agent-starter-anthropic`	`ANTHROPIC_API_KEY`
Google Gemini	`embabel-agent-starter-gemini`	`GEMINI_API_KEY`
Google GenAI	`embabel-agent-starter-google-genai`	`GOOGLE_API_KEY`
DeepSeek	`embabel-agent-starter-deepseek`	`DEEPSEEK_API_KEY`
Mistral AI	`embabel-agent-starter-mistral-ai`	`MISTRAL_API_KEY`
Ollama	`embabel-agent-starter-ollama`	Local server
LM Studio	`embabel-agent-starter-lmstudio`	Local server
AWS Bedrock	`embabel-agent-bedrock-autoconfigure`	AWS credentials

7.7 LLM Reasoning / Thinking

Access LLM reasoning with the `thinking()` API:

```
PromptRunner runner = context.ai()
    .withLlm("claude-sonnet-4-5")
    .withToolObject(Tooling.class);

ThinkingResponse<MonthItem> response = runner.thinking()
    .createObject("What is the hottest month in Florida?", MonthItem.class);

// Access the structured result
MonthItem result = response.getResult();

// Access the LLM's reasoning process
List<ThinkingBlock> thinkingBlocks = response.getThinkingBlocks();
for (ThinkingBlock block : thinkingBlocks) {
    System.out.println("Content: " + block.getContent());
}
```

---

8. State Management

8.1 The @State Annotation

The `@State` annotation enables state-based workflows within GOAP planning:

```
@Agent(description = "Multi-step document processor")
public class DocumentProcessor {

    @Action(clearBlackboard = true)
    public ProcessedDocument preprocess(RawDocument doc) {
        return new ProcessedDocument(doc.getContent().trim());
    }

    @AchievesGoal(description = "Produce final output")
    @Action
```

```
    public FinalOutput transform(ProcessedDocument doc) {
        return new FinalOutput(doc.getContent().toUpperCase());
    }
}
```

8.2 State Class Requirements

State classes must be either:

- **Static nested classes** (Java records are implicitly static)
- **Top-level classes**

```
// GOOD: Static nested class
@State
record AssessStory(UserInput userInput, Story story) implements Stage { ... }

// GOOD: Top-level class
@State
record ProcessingState(String data) { ... }
```

8.3 Looping States

Use `clearBlackboard = true` for looping patterns:

```
@State
record ProcessingState(String data, int iteration) implements LoopOutcome {

    @Action(clearBlackboard = true)
    LoopOutcome process() {
        if (iteration >= 3) {
            return new DoneState(data);
        }
        return new ProcessingState(data + "+", iteration + 1);
    }
}
```

8.4 Parent State Interface Pattern

Define a parent interface for polymorphic state returns:

```
@State
interface Stage {}

record AssessStory(String content) implements Stage {
    @Action
    Stage assess() {
        if (isAcceptable()) {
            return new Done(content);
        } else {
            return new ReviseStory(content);
        }
    }
```

```
}

record ReviseStory(String content) implements Stage {
    @Action
    AssessStory revise() {
        return new AssessStory(improvedContent());
    }
}

record Done(String content) implements Stage {
    @AchievesGoal(description = "Processing complete")
    @Action
    Output complete() {
        return new Output(content);
    }
}
```

8.5 Human-in-the-Loop with WaitFor

```
@State
record AssessStory(UserInput userInput, Story story, Properties properties) implements Stage {

    @Action
    HumanFeedback getFeedback() {
        return WaitFor.formSubmission("""
            Please provide feedback on the story
            %s
            """.formatted(story.text), HumanFeedback.class);
    }

    @Action(clearBlackboard = true)
    Stage assess(HumanFeedback feedback, Ai ai) {
        var assessment = ai.withDefaultLlm()
            .createObject("Is this story acceptable? " + feedback.comments(),
                          AssessmentOfHumanFeedback.class);
        if (assessment.acceptable()) {
            return new Done(userInput, story, properties);
        } else {
            return new ReviseStory(userInput, story, feedback, properties);
        }
    }
}
```

---

9. Planning Strategies

9.1 GOAP (Goal-Oriented Action Planning)

GOAP is the default planner. It plans a path from current state to goal using preconditions and effects:

```
@Agent(description = "Business process agent", planner = PlannerType.GOAP)
```

```
public class BusinessProcessAgent {
    // Actions with preconditions and effects
}
```

Best for: Business processes with defined outputs

9.2 Utility AI

Utility AI selects the highest-value available action at each step:

```
@EmbabelComponent
public class IssueActions {

    @Action(cost = 0.1, value = 0.8)
    public Output highValueAction(Input input) {
        // Action implementation
    }
}

// Create utility-based chatbot
@Bean
Chatbot chatbot(AgentPlatform agentPlatform) {
    return AgentProcessChatbot.utilityFromPlatform(agentPlatform);
}
```

Best for: Exploration and event-driven systems

9.3 Supervisor

Supervisor uses an LLM to orchestrate actions dynamically:

```
@Agent(description = "Market research agent", planner = PlannerType.SUPERVISOR)
public class MarketResearchAgent {

    @Action(description = "Gather market data")
    public MarketData gatherMarketData(MarketDataRequest request, Ai ai) {
        return ai.withDefaultLlm()
            .createObject("Generate market data for: " + request.topic(),
                          MarketData.class);
    }

    @AchievesGoal(description = "Compile final report")
    @Action
    public FinalReport compileReport(ReportRequest request, Ai ai) {
        return ai.withDefaultLlm()
            .createObject("Create market research report", FinalReport.class);
    }
}
```

Best for: Flexible multi-step workflows

9.4 Choosing a Planner

Planner	Best For	Deterministic
GOAP	Business processes with defined outputs	Yes
Utility	Exploration and event-driven systems	Yes
Supervisor	Flexible multi-step workflows	No

---

10. RAG and Context Engineering

10.1 ToolishRag

`ToolishRag` exposes search operations as LLM tools:

```
@EmbabelComponent
public class ChatActions {
    private final ToolishRag toolishRag;

    public ChatActions(SearchOperations searchOperations) {
        this.toolishRag = new ToolishRag("sources", "Sources for answering questions",
                                         searchOperations);
    }

    @Action(canRerun = true, trigger = UserMessage.class)
    void respond(Conversation conversation, ActionContext context) {
        var assistantMessage = context.ai()
            .withLlm(properties.chatLlm())
            .withReference(toolishRag)
            .rendering("ragbot")
            .respondWithSystemPrompt(conversation, Map.of("properties", properties));
        context.sendMessage(conversation.addMessage(assistantMessage));
    }
}
```

10.2 Search Operations

Interface	Capability
`VectorSearch`	Semantic vector search
`TextSearch`	Full-text search using Lucene query syntax
`ResultExpander`	Expand search results to surrounding context
`RegexSearchOperations`	Pattern-based search using regular expressions
`CoreSearchOperations`	Convenience interface combining vector and text search

10.3 Document Ingestion

```
@ShellMethod("Ingest URL or file path")
String ingest(@ShellOption(defaultValue = "./data/document.md") String location) {
```

```
    var uri = location.startsWith("http://") || location.startsWith("https://")
        ? location
        : Path.of(location).toAbsolutePath().toUri().toString();

    var ingested = NeverRefreshExistingDocumentContentPolicy.INSTANCE
        .ingestUriIfNeeded(luceneSearchOperations,
                           new TikaHierarchicalContentReader(),
                           uri);

    return ingested != null
        ? "Ingested document with ID: " + ingested
        : "Document already exists";
}
```

10.4 RAG Stores

Store	Module	Capabilities
Lucene	`embabel-agent-rag-lucene`	Vector, text, regex search, result expansion
Neo4j	`embabel-agent-rag-neo-drivine`	Graph database with vector search
PostgreSQL pgvector	`embabel-rag-pgvector`	Hybrid search (vector + full-text + fuzzy)
Spring AI VectorStore	Built-in	Adapter for any Spring AI VectorStore

10.5 Filtered RAG

```
// Filter on metadata (e.g., which user owns the document)
PropertyFilter metadataFilter = PropertyFilter.eq("ownerId", currentUserId);

// Filter on entity labels and properties
EntityFilter entityFilter = EntityFilter.hasAnyLabel("Person");

// Apply both filters
ToolishRag scopedRag = toolishRag
    .withMetadataFilter(metadataFilter)
    .withEntityFilter(entityFilter);
```

---

11. Building Chatbots

11.1 Core Concepts

Chatbots in Embabel are backed by long-lived AgentProcesses that pause between user messages:

```
@Configuration
class ChatConfiguration {

    @Bean
    Chatbot chatbot(AgentPlatform agentPlatform) {
        return AgentProcessChatbot.utilityFromPlatform(agentPlatform);
```

```
    }
}
```

11.2 Chat Actions

```
@EmbabelComponent
public class ChatActions {
    private final ToolishRag toolishRag;
    private final RagbotProperties properties;

    public ChatActions(SearchOperations searchOperations, RagbotProperties properties) {
        this.toolishRag = new ToolishRag("sources", "Sources for answering questions",
                                         searchOperations);
        this.properties = properties;
    }

    @Action(canRerun = true, trigger = UserMessage.class)
    void respond(Conversation conversation, ActionContext context) {
        var assistantMessage = context.ai()
            .withLlm(properties.chatLlm())
            .withReference(toolishRag)
            .rendering("ragbot")
            .respondWithSystemPrompt(conversation, Map.of("properties", properties));
        context.sendMessage(conversation.addMessage(assistantMessage));
    }
}
```

11.3 Using the Chatbot

```
// New session
ChatSession session = chatbot.createSession(user, outputChannel, null, null);

// Send a user message
session.onUserMessage(new UserMessage("What does this document say about taxes?"));
```

11.4 Conversation Storage

```
# In-memory (default)
embabel:
  chat:
    store:
      type: IN_MEMORY

# Persistent storage
embabel:
  chat:
    store:
      type: STORED
```

---

12. Testing

12.1 Unit Testing with FakeOperationContext

```
class WriteAndReviewAgentTest {

    @Test
    void testWriteAndReviewAgent() {
        var context = FakeOperationContext.create();
        var promptRunner = (FakePromptRunner) context.promptRunner();
        context.expectResponse(new Story("Once upon a time Sir Galahad..."));

        var agent = new WriteAndReviewAgent(200, 400);
        agent.craftStory(new UserInput("Tell me a story about a brave knight",
                                       Instant.now()), context);

        String prompt = promptRunner.getLlmInvocations().getFirst().getPrompt();
        assertTrue(prompt.contains("knight"), "Expected prompt to contain 'knight'");

        var temp = promptRunner.getLlmInvocations().getFirst()
                               .getInteraction().getLlm().getTemperature();
        assertEquals(0.9, temp, 0.01, "Expected temperature to be 0.9");
    }
}
```

12.2 Integration Testing

```
@SpringBootTest
class WriteAndReviewAgentIntegrationTest extends EmbabelMockitoIntegrationTest {

    @Autowired
    private AgentPlatform agentPlatform;

    @Test
    void testCompleteWorkflow() {
        whenCreateObject(prompt -> prompt.contains("Craft a short story"), Story.class)
            .thenReturn(new Story("AI will transform our world..."));

        var invocation = AgentInvocation.create(agentPlatform, ReviewedStory.class);
        var result = invocation.invoke(new UserInput("Tell me a story"));

        assertNotNull(result);
        verifyCreateObjectMatching(
            prompt -> prompt.contains("Craft a short story"),
            Story.class,
            llm -> llm.getLlm().getTemperature() == 0.7
        );
    }
}
```

12.3 Testing Multiple LLM Interactions

```
@Test
void shouldHandleMultipleLlmInteractions() {
    // Set up expected responses in order
    context.expectResponse(story);
    context.expectResponse(review);

    // Act
    var writtenStory = agent.writeStory(input, context);
    ReviewedStory reviewedStory = agent.reviewStory(writtenStory, context);

    // Verify both LLM calls were made
    List<LlmInvocation> invocations = context.getLlmInvocations();
    assertEquals(2, invocations.size());
}
```

---

13. Deployment and Integration

13.1 Invoking Agents Programmatically

```
// Using AgentInvocation
var invocation = AgentInvocation.create(agentPlatform, TravelPlan.class);
TravelPlan plan = invocation.invoke(travelRequest);

// Async invocation
CompletableFuture<TravelPlan> future = invocation.invokeAsync(travelRequest);
```

13.2 Web Application Integration

```
@Controller
public class TripPlanningController {
    private final AgentPlatform agentPlatform;

    @PostMapping("/plan-trip")
    public String planTrip(@ModelAttribute TripRequest tripRequest, Model model) {
        String jobId = UUID.randomUUID().toString();

        var processOptions = new ProcessOptions()
            .withVerbosity(new Verbosity().withShowPrompts(true));

        var invocation = AgentInvocation.builder(agentPlatform)
            .options(processOptions)
            .build(TripPlan.class);

        CompletableFuture<TripPlan> future = invocation.invokeAsync(tripRequest);
        activeJobs.put(jobId, future);

        model.addAttribute("jobId", jobId);
        return "trip-planning-progress";
```

```
    }
}
```

13.3 MCP Server Publishing

Expose agents as MCP servers:

```
@Agent(description = "Weather service agent")
public class WeatherAgent {

    @AchievesGoal
    @Export(remote = true)  // Automatically becomes MCP tool
    @Action
    public WeatherInfo getWeather(@Param("location") String location,
                                  @Param("units") String units) {
        return weatherService.getWeather(location, units);
    }
}
```

13.4 Observability

Add the observability starter:

```
<dependency>
    <groupId>com.embabel.agent</groupId>
    <artifactId>embabel-agent-starter-observability</artifactId>
</dependency>
```

Configure in `application.yml`:

```
embabel:
  observability:
    enabled: true
    service-name: my-agent-app
```

---

14. Advanced Topics

14.1 Streaming Responses

```
PromptRunner runner = context.ai().withLlm("qwen3:latest");

Flux<StreamingEvent<MonthItem>> results = new StreamingPromptRunnerBuilder(runner)
    .withStreaming()
    .withPrompt("What are the hottest months in Florida?")
    .createObjectStreamWithThinking(MonthItem.class);

results
    .doOnNext(event -> {
        if (event.isThinking()) {
            System.out.println("THINKING: " + event.getThinking());
```

```
        } else if (event.isObject()) {
            System.out.println("OBJECT: " + event.getObject().getName());
        }
    })
    .blockLast(Duration.ofSeconds(6000));
```

14.2 Guardrails

```
class CriticalUserInputGuardRail implements UserInputGuardRail {
    @Override
    public ValidationResult validate(String input, Blackboard blackboard) {
        if (containsSensitiveData(input)) {
            return new ValidationResult(true, List.of(
                new ValidationError("sensitive-data",
                    "Input contains sensitive data",
                    ValidationSeverity.CRITICAL)
            ));
        }
        return ValidationResult.VALID;
    }
}

// Use with PromptRunner
PromptRunner runner = context.ai()
    .withLlm("claude-sonnet-4-5")
    .withGuardRails(new CriticalUserInputGuardRail())
    .withToolObject(Tooling.class);
```

14.3 Custom LLM Integration

Implement `LlmMessageSender` for custom LLM providers:

```
public class MyCustomLlmMessageSender implements LlmMessageSender {

    @Override
    public LlmMessageResponse call(List<Message> messages, List<Tool> tools) {
        // Convert messages to your API's format
        // Make API request
        // Convert response to LlmMessageResponse
        return new LlmMessageResponse(message, textContent, usage);
    }
}
```

14.4 Agentic Tools

```
// SimpleAgenticTool - All tools available immediately
SimpleAgenticTool mathOrchestrator = new SimpleAgenticTool("math-orchestrator",
    "Orchestrates math operations")
    .withTools(addTool, multiplyTool, divideTool)
    .withParameter(Tool.Parameter.string("expression", "Math expression to evaluate"))
    .withLlm(LlmOptions.withModel("gpt-4"));
```

```
// PlaybookTool - Progressive unlock via conditions
PlaybookTool researcher = new PlaybookTool("researcher", "Research topics")
    .withTools(searchTool, fetchTool)
    .withTool(analyzeTool).unlockedBy(searchTool)
    .withTool(summarizeTool).unlockedBy(analyzeTool);

// StateMachineTool - State-based availability
StateMachineTool<OrderState> orderProcessor = new StateMachineTool<>("orderProcessor",
    "Process orders", OrderState.class)
    .withInitialState(OrderState.DRAFT)
    .inState(OrderState.DRAFT)
        .withTool(addItemTool)
        .withTool(confirmTool)
        .transitionsTo(OrderState.CONFIRMED);
```

---

15. Configuration Reference

15.1 Core Configuration Properties

```
embabel:
  agent:
    platform:
      name: embabel-default
      description: Embabel Default Agent Platform

      # Agent scanning
      scanning:
        annotation: true  # Auto-register @Agent and @Agentic
        bean: false       # Auto-register beans of type Agent

      # Autonomy configuration
      autonomy:
        agent-confidence-cut-off: 0.6
        goal-confidence-cut-off: 0.6

      # LLM operations
      llm-operations:
        prompts:
          generate-examples-by-default: true
        data-binding:
          max-attempts: 10
          fixed-backoff-millis: 30

      # HTTP client
      http-client:
        connect-timeout: 25s
        read-timeout: 1m
```

```yaml
# Model configuration
models:
  default-llm: gpt-4o-mini
  default-embedding-model: text-embedding-3-small
  llms:
    cheapest: gpt-4o-mini
    best: gpt-4o
  embedding-services:
    fast: text-embedding-3-small
```

15.2 Logging Personalities

```yaml
embabel:
  agent:
    logging:
      personality: starwars  # Options: starwars, severance, colossus, hitchhiker, montypython
```

15.3 Provider-Specific Configuration

OpenAI:

```yaml
embabel:
  agent:
    platform:
      models:
        openai:
          api-key: ${OPENAI_API_KEY}
          base-url: ${OPENAI_BASE_URL:}
          max-attempts: 10
          backoff-millis: 5000
```

Anthropic:

```yaml
embabel:
  agent:
    platform:
      models:
        anthropic:
          api-key: ${ANTHROPIC_API_KEY}
          base-url: ${ANTHROPIC_BASE_URL:}
          max-attempts: 10
          backoff-millis: 5000
```

---

16. Best Practices

16.1 Design Principles

1. **Start Simple**: Begin with simple agents and add complexity incrementally
2. **Use Strong Typing**: Leverage Java/Kotlin's type system for reliable data flow
3. **Test Early and Often**: Use FakeOperationContext for unit testing

4. **Separate Concerns**: Use @EmbabelComponent for shared actions
5. **Choose the Right Planner**: GOAP for defined goals, Utility for exploration, Supervisor for flexibility

16.2 Prompt Engineering

1. **Use Jinja Templates**: For complex prompts, use templates instead of inline strings
2. **Provide Examples**: Use `withExample()` to improve LLM output quality
3. **Be Specific**: Clear instructions produce better results
4. **Iterate**: Test different prompts and measure results

16.3 Tool Design

1. **Keep Tools Focused**: Each tool should do one thing well
2. **Use Descriptive Names**: Help the LLM understand tool purpose
3. **Document Parameters**: Use @LlmTool.Param descriptions
4. **Consider Tool Groups**: Organize related tools together

16.4 Performance Optimization

1. **Mix LLMs**: Use smaller models for simple tasks, larger models for complex ones
2. **Use Eager Search**: Preload RAG results to reduce latency
3. **Configure Timeouts**: Adjust HTTP timeouts for your use case
4. **Monitor Costs**: Use observability to track token usage

16.5 Security Considerations

1. **Use Guardrails**: Validate inputs and outputs
2. **Filter RAG Results**: Apply metadata filters for multi-tenant scenarios
3. **Secure API Keys**: Use environment variables or secret management
4. **Audit Actions**: Use @Tracked for observability

Resources

Official Resources

- **Documentation**: docs.embabel.com
- **API Docs**: docs.embabel.com/embabel-agent/api-docs
- **Examples**: github.com/embabel/embabel-agent-examples
- **Java Template**: github.com/embabel/java-agent-template
- **Kotlin Template**: github.com/embabel/kotlin-agent-template

Community

- **Discord**: Active community for support and collaboration
- **GitHub Issues**: Bug reports and feature requests
- **GitHub Discussions**: Community discussions and Q&A

From the creator of Spring Framework - bringing enterprise-grade agent development to the JVM ecosystem.